Storage Options
==========

Excel Files	databases
data is stored in the form of sheets	data is stored in the form of tables
limited support of relationships b/w sheets	strong supports relationships b/w tables
around 1 million rows/sheet (10 lakhs) (2-5 GB total)	there is a size limit (depends on platform)(TB to PB)
slow performance if size 100+ MB	smooth execution
poor support of data integrity	strong support & validation rules
default column headers are not interpretable(A,B,C,..)	we have to specify column names by own
limited support for multi-user environment	strong support for multi-user environment
less secure	highly secure(role based)
limited support of automation (macros,formules)	strong support of automation (triggers)
supports visualization	does not support visualization

RDBMS(Relational Database Management System)
==========
>It is a software package that combines databases & other utilities
>these are the following vendors who created RDBMS

Product		Vendor
MS SQL Server	by	Microsoft
MySQL Server	by	Sun Microsystem(now merged in Oracle corp.)
DB2 Server	by	IBM
Postgre Server	by	Postgre org

OracleDB Server by Oracle corp.

etc.

we may download c++ dependency for mysql from
https://aka.ms/vs/17/release/vc_redist.x64.exe
after downloading it,install it then download mysql server from
https://dev.mysql.com/downloads/mysql/5.5.html?os=3&version=5

SQL(Structured Query Language)
==========

>It is common language for all RDBM softwares that perform all operations related to structured data like storing,fetching,filtering,manipulation,organizing,etc.

>It is used in multiple fields like in development,analyst,etc.

>It is a command based language and is not case sensitive.

>SQL standard is maintained by ANSI/ISO and different rdbms follow it with some variations.

MySQL Editors:
---

1. Command Line Interface(CLI)/terminal
2. Workbench(GUI)

how to create database

```
create database_name;
```

how to show databases

```
show databases;
```

how to active a database

```
use database_name;
```

how to create table

```
create table table_name(col_name datatype,col_name datatype,.....);
```

how to insert records:
---

insert into table_name values(value1,value2,....);
insert into table_name values(value1,value2,...),(value1,value2,...),.....;

Note:we must insert same no. of values as no. of columns

with missing value(null):
---
insert into table_name values(value1,null,value2,...);

how to fetch records:
---

with specific cols
---
select col1,col2,... from table_name;

with all cols
---
select * from table_name;

with limit to fetch 1st N records
---
select * from table_name limit N;

with limit & offset
---
select * from table_name limit N offset K;

offset-->skip first K records

with order by
---
select * from table_name order by col asc/desc;

Note:
>if we do not specify order,default is asc
>we may also use 2 or more cols with order by

select * from table_name order by col1 asc/desc , col2 asc/desc ,....;

Note:col2 is used only for those rows where values are same in col1

with order by & limit
---
select * from table_name order by asc/desc limit N;

with where to filter records based on condition
---
select * from table_name where condition;

etc.

Operators:
=========
>These are special symbols and sometimes keywords also
>represent mathematical,comparison,logical operations with data

comparison Operators

=	equal comparison of values
is	equal comparison of null values
!=	not equal comparison of values
<>	not equal comparison of values
is not	not equal comparison of null values
>	greater comparison of values
>=	greater or equal comparison of values
<	lesser comparison of values
<=	lesser or equal comparison of values
in	compare more than 1 values in same column
not in	compare more than 1 values in same column in not equal manner
between	compare values in the given range(both inclusive) select ..... where col between start and stop;
like	compare values based on pattern %---------->0 to N chars _---------->1 char exp: select .... name like 'a%'; will match records where name starts with 'a' select .... name like '%a'; will match records where name ends with 'a' select .... name like '%a%'; will match records where name contains 'a' select .... name like 'a__'; will match records where name starts with 'a' then exact 2 chars

not like	self

Logical operators

>combine multiple conditions to filter records

and	match records if both conditions are True
or	match records if either condition is True

Arithmetic operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder

Assignment operator
---

set =	Assign value

etc.

How to update records
==========

update all records for 1 col

```
update table_name set col=expression;
```

update first N records for 1 col

```
update table_name set col=expression limit N;
```

update all records for 1 col with order by

```
update table_name set col=expression order by col asc/desc;
```

update only filtered records for 1 col

```
update table_name set col=expression where condition;
```

update only filtered records for 2 cols

```
update table_name set col1=expression,col2=expression where condition;
```

update multiple records with different update rule

```
update table_name set col=case
when condition1 then expression1
when condition2 then expression2
.
.
else expression
end;
```

How to delete records(rows)
==========

delete command is used to delete 0-N records

to delete all records

```
delete from table_name;
```

to delete specific records

```
delete from table_name where condition;
```

to delete first N records;

```
delete from table_name limit N;
```

to delete records in specific order

```
delete from table_name order by col asc/desc;
```

truncate command
==========

>it is used to delete all records from a table in faster manner than delete command

```
truncate table table_name
```

Note:truncate does not support where,limit,order by,rollback options.

drop commnad
==========

>it is used to drop any created database entity

drop entity entity_name;

drop table table_name;
drop database database_name;
.
.

alter command
==========

>it is used to modify schema of database entity

to add a column
---
alter table table_name add column col_name datatype; #added to last

to add a column after specific col:
---
alter table table_name add column col_name datatype after exit_col;

to add a column first
---
alter table table_name add column col_name datatype first;

to drop a column
---
alter table table_name drop column col_name;

to modify datatype of a column
---
alter table table_name modify column col_name new_datatype;

to rename a column
---
alter table table_name rename column col_name to new_col_name;

to rename table
---
alter table table_name rename to new_table_name;

Constraints:

==========

>A constraint defines validation rule that is checked before executing insert/update/delete commands and if rule fails,command is terminated by raising error.

>There are following constraints in MySQL:

1.unique
2.not null
3.primary key
4.foreign key
5.composite key
6.check

UNIQUE CONSTRAINT:

>It allows only unique values to be inseted in applied column

Note:duplicate NULL values are allowed

syntax with create:
---

column level:

create table table_name (col_name datatype unique,col_name datatype,...)

table level:

create table table_name(col_name datatype,...,constraint identifier unique(col_name));

Note:if we don't provide identifier,it is given by sql itself and we can check it use following command

show create table table_name;

syntax with alter:
---

alter table table_name add constraint identifier unique(col_name);

syntax to drop:
---

alter table table_name drop constraint identifier;

NOT NULL:

>it will not allow missing(null) values to be inserted in the applied column

Note:It is available in column level form only.

syntax with alter:

---

alter table table_name modify column col_name datatype not null;

syntax with create:

---

create table table_name(col_name datatype not null,col_name datatype unique not null,.....);

syntax to drop:

---

alter table table_name modify column datatype;

PRIMARY KEY:

>A primary key is used to identify each record uniquely in a table

>No two rows can have the same primary key value

>It cannot be empty (NULL)

>A table can have only one primary key

syntax with alter:

---

alter table table_name add constraint primary key(col_name);

syntax with create:

---

create table table_name(col_name datatype primary key,.....);

syntax to drop:

---

alter table table_name drop primary key; #it will remove only unique part

alter table table_name modify column col_name datatype; #it will remove not null part

COMPOSITE KEY

>It is actually a primary or unique key that is defined using multiple cols collectively.

>Individual cols used in composite key can have duplicate values but not collectively.

>It is always defined as table level constraint.

syntax with create:
---
create table table_name(col_1 datatype,col_2 datatype,....,primary key(col1,col2,..));

syntax with alter:
---
alter table table_name add primary key(col1,col2,...);

syntax to drop:
---
alter table table_name drop primary key;
also drop not null from participating cols

foreign key constraint:

>It is used to create relationship between 2 tables(1 is parent and other is child)
>It means insertion,deletion,updation operations are related to each other not independent.

syntax with create:
---
1. create table child_table(col_name datatype,.....,foreign key(col_name)
references parent_table(PK_col)) ;

2. create table child_table(col_name datatype,.....,constraint constraint_name foreign key
(col_name) references parent_table(PK_col)) ;

syntax to drop:
---
alter table table_name drop constraint constraint_name;

syntax with alter:
---
alter table child_table add constraint constraint_name foreign key
(col_name) references parent_table(PK_col)

with update/delete cascade

>provide facility to delete or update a child row when it is updated or delete in parent

alter table child_table add constraint constraint_name foreign key
(col_name) references parent_table(PK_col) on update cascade on delete cascade

CHECK CONSTRAINT

>It is used to validate custom rules before inserting & updating records
exp: age>0,marks between 0 and 100,gender must be in male or female,etc.

>It is always applied as table level

syntax with create:
---
create table table_name(col_1 datatype,col_2 datatype,.....,constraint name check(rule));

syntax with alter:
---
alter table table_name add constraint name check(rule);

syntax to drop:
---
alter table table_name drop constraint name;

Column Properties
==========

>A property provides additional feature to applied column
>There are 2 commonly used col properties
1.auto_increment
2.default

AUTO_INCREMENT PROPERTY:

>It is used to automatically insert a value while inserting a record when the value is not specified or is NULL.
>New value is inserted according to following rule
if the table is empty-->starts from 1
else---------->max_value+1

>It can be used with integer type column with either unique or primary key constraint
>Only a single auto_increment property is allowed per table.
>We may also insert a value in applied column.
>AUTO_INCREMENT values are not reused after DELETE; gaps are expected and

normal.

>If you truncate table then only it will start from 1

syntax with alter:

---

alter table table_name modify column col_name datatype auto_increment;

syntax with create:

---

create table table_name(col_name int primary key auto_increment,col_name datatype,...);

syntax to drop:

---

alter table table_name modify column col_name datatype;

syntax to change next value:

---

alter table table_name auto_increment=next_start_value;

DEFAULT PROPERTY:

>It is used to specify a fixed default value that will be inserted for each row when not given

syntax with alter:

---

alter table table_name modify column col_name datatype default default_value;

syntax with create:

---

create table table_name(col_name datatype default default_value,...);

syntax to drop:

---

alter table table_name modify column col_name datatype;

Transaction:

==========

>A transaction is a set of logically related dml queries that are either committed or rolled back.

>It works for DML commands only.

syntax to start txn:
---
start transaction;

DML1
DML2
.
.
rollback;

>rollback command is used to null & void all non committed DML operations executed within transaction and end the txn.

>commit command is used to persist all DML operations and end the transaction.

Note:
>if you run DDL commands within transaction then all commands will be committed and also
transaction will be completed(MySQL specific).

Transaction with savepoint:

>we may also create savepoints to partially rollback the commands

```
start transaction;
        savepoint s1;
                DML1
                DML2
                ..
        savepoint s2;
                DML1
                DML2
                ..
        savepoint s3;
                DML1
                DML2
                ..
        rollback to s2;
```

>this rollback command will null & void all DML commands after creating savepoint s2(including all further savepoints) but transaction remains exist .

Note:

>in case of system failure,transaction will be rolled back automatically.

>commit is not applicable with savepoint

MySQL Index:

==========

>Indexes are used to retrieve data from the table more quickly than otherwise.

>The users cannot see the indexes, they are just used to speed up select queries with applied column.

Syntax to create index:

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

Syntax to drop index:

```
ALTER TABLE table_name DROP INDEX index_name;
```

To show all indexes on a table:

```
SHOW INDEXES FROM table_name;
```

Downside of Index:

>Extra memory is required to maintain key-value in a separate table

>Slow down execution of insert,delete & update queries as index table also need to be updated

>It means,we should create index on a table when have done with all DML operations and only select operations are now required.

Types of Index:

---

1. Clustered or Primary key Index
   - >Automatically created on primary key column
   - >sort the records of existing table
   - >only 1 clustered index is allowed on a table
2. Non-Clustered or Non-PK Indexes
   - >other indexes are called non-clustered indexes
   - >we can create,drop these indexes according to requirement

Data types:

=========

provide following 3 informations about values:

1.nature(numeric,text,date,etc.),it helps to decide operations
2.range(min and max)
3.memory usage

1.tinyint
    1 byte/value -128 to +127

2.smallint
    2 bytes/value -32768 to +32767

3.mediumint
    3 bytes/value -8388608 to +8388607

4.int
    4 bytes/value -2147483648 to +2147483647

5.bigint
    8 bytes/value -9223372036854775808 to +9223372036854775807

6.float
    4 bytes/value total digits=7,decimal places=4 (approx.)

7.double
    8 bytes/value total digits=16,decimal places=8 (approx.)

8.decimal(M,D)
    dynamic size M=total digits,D=decimal places
    (general hint,for every 9 digits,it takes 4 bytes)

    default values
        D=0
        M=10
    Max M=65
    Max D=30

9.char(length)
    1-4 bytes/char
    255 max length
    default 1 char

10.varchar(length)

1-4 bytes/char
16383 max length
no default size

diff b/w char(10) & varchar(10)
---

char(10)	varchar(10)
memory will be allocated to exact 10 chars.(spaces are appended if value is less than 10 chars.)	memory is allocated to only given chars.(no extra spaces are added)
max size=255 chars	max size=16383 chars
default length is 1 char	no default length

11.tinytext
>variable length
>max(default) 255 chars
>1-4 byte/char
12.text
>variable length
>max(default) 65535 chars
>1-4 byte/char
13.mediumtext
>variable length
>max 16,777,215(default) chars (16 MB)
>1-4 byte/char
14.longtext
>variable length
>max 4,294,967,295(default) chars(4GB)
>1-4 bytes/char
Note:
text datatypes do not support default property

15.date
default format yyyy-mm-dd
'0000-01-01' to '9999-12-31'
>3 bytes/date

Note:

Year values in the range 00-69 become 2000-2069.
Year values in the range 70-99 become 1970-1999.

16. time

default format hh:mm:ss (24 hrs format)
'00:00:00' to '23:59:59'
>3 bytes/time

17. datetime

default format yyyy-mm-dd hh:mm:ss
'0000-01-01 00:00:00' to '9999-12-31 23:59:59'
>5 bytes/datetime

18. timestamp

default format yyyy-mm-dd hh:mm:ss
'1970-01-01 00:00:01' to '2038-01-19 03:14:07'
>4 bytes/timestamp

Note:tinyblob,blob,mediumblob,longblob datatypes are used to store binary files(images,A/V,etc.)

memory size

tinyblob	255 bytes
blob	65535 bytes
mediumblob	16,777,215 bytes (16 MB)
longblob	4,294,967,295 bytes (4 GB)
etc.	

Combining result from multiple tables:
==========

1.Union(row wise)
2.Join(column wise)

Union:

>it is best suitable when tables are having same schemas
syntax:

```
select col1,col2,... from table1
union
select col1,col2,....from table2
```

union
select col1,col2,....from table3
.
.

Note:

>union will keep only first occurrence of duplicate rows and if you want to keep all occurrences
use union all

>used select statements must have same num of columns

>result will contain columns according to first select statement

Join:

>it is best suitable when there must be at least 1 matching column

>there are following types of join

1.inner join
2.left join
3.right join
4.full(outer) join
5.natural join
6.cross join
7.self join

Inner Join:

>it returns matching rows based on given criteria

basic syntax:

select * from table1 inner join table2 on criteria;
select * from table1 inner join table2 on criteria inner join table3 on criteria;

>if we want to use specific column name then it is a good practice to use col name with table

exp:

select table1.col,table1.col,table2.col.....

Aliasing:

>it is a process of giving temporary name for a column or table

column aliasing:

select column as alias from table

exp:

select pnb_adhar as Adhar from pnb;

table aliasing:

select table_alias.column from table as table_alias

exp:

select pnb.bal from punjabnationalbank as pnb;

View:

>A view is a virtual table that refers to other tables and generally we use views for frequently used select queries.

>most of the time,complex joins are implemented by defining views

syntax:

---

create view view_name as select query;

>we may now run select,delete,update queries on view and view will actually perform operation with referenced tables.

syntax:

---

select * from view_name;

Natural Join:

>it is a special case of inner join that works without on

>it automatically fetch matched rows according to same name columns

>in the result,same name columns will appear only once if used with *

syntax:

select * from table1 natural join table2;

Note:inner join gives more control over query than natural join

Left Join:

>it returns all records from 1st(left) table and only matching from 2nd(right)

syntax:

```
select * from left_table left join right_table on criteria;
```

Right Join:

>it returns all records from 2nd(right) table and only matching from 1st(left)

syntax:

```
select * from left_table right join right_table on criteria;
```

Full(Outer) Join:

>it returns all records matching & non-matching from both tables

>MySQL does not support outer join directly and we will take union of left & right joins

syntax:

```
select * from left_table left join right_table on criteria
union
select * from left_table right join right_table on criteria;
```

Cross Join:

>it returns records from both tables according to cartesian product(in maths)

syntax-1:

```
select * from table1 cross join table2;
```

syntax-2:

```
select * from table1 join table2;
```

syntax-3:

```
select * from table1,table2;
```

syntax-4:

```
select * from table1 inner join table2;
```

Self Join:

>it is not a different category of joins rather it provides facility to implement any join on a single table by creating multiple aliases for same table.

>it means,self join technique can be used to implement inner,left,right,cross joins

>it is best suitable to find duplicate records in a table

syntax with cross join:
---
select * from table as alias1,table as alias2;

MySQL Functions:
==========
>A function represents piece of readymade code that can be reused on data.
>MySQL provides set of functions:

Numeric Functions:
---
round()
float_value=round(float_value,decimal_places)
or
int_value=round(float_values)
.5 or above--->next int
<.5 ---------->same int

with column:
---
select round(col_name) from table_name;

update table_name set col_name=round(col_name);

sqrt()---->returns square root of given value or column

result=sqrt(value | col_name)

Note:NULL if value is -ve

pow()-->returns a raise to power b
result=pow(a,b)

abs()-->returns unsigned value
unsigned_value=abs(value)

etc.

String:

CHAR_LENGTH	Returns the length of a string (no. of characters)
LENGTH	Returns the size of memory(no. of bytes)
CONCAT	Adds two or more expressions together
LOWER	Converts a string to lower-case
REVERSE	Reverses a string and returns the result
UPPER	Converts a string to upper-case
LEFT	left("india",3)-->ind
RIGHT	right("india",3)-->dia
MID	mid("india",2,3)-->nd
TRIM	remove spaces from both ends
LTRIM	remove spaces from front end
RTRIM	remove spaces from back end

Date/Time:

---

curdate()	Returns the current date
curtime()	Returns the current time
now()	Returns the current date and time
day()	used to get the day from the given date.
dayname()	used to get the name of the day from the given date.
dayofweek()	used to get the day of the week in numeric.
dayofyear()	used to get the number of day in the year.
hour()	used to get the hour from the given datetime.
month()	used to get the value of month from given datetime or time.
monthname()	used to get the full month name.
week()	used to get the week portion for the specified date.
weekofyear()	used to get the week of the given date.
date_add()	date_add("yyyy--mm--dd",interval n unit)

#units

MICROSECOND
SECOND
MINUTE
HOUR
DAY
WEEK
MONTH
QUARTER
YEAR

datediff() returns diff b/w two dates in days

DATE_FORMAT(date, format)

Format	Description
%a	Abbreviated weekday name (Sun to Sat)
%b	Abbreviated month name (Jan to Dec)
%c	Numeric month name (0 to 12)
%D	Day of the month as a numeric value, followed by suffix (1st, 2nd, 3rd, ...)
%d	Day of the month as a numeric value (01 to 31)
%e	Day of the month as a numeric value (0 to 31)
%f	Microseconds (000000 to 999999)
%H	Hour (00 to 23)
%h	Hour (00 to 12)
%I	Hour (00 to 12)
%i	Minutes (00 to 59)
%j	Day of the year (001 to 366)
%k	Hour (0 to 23)
%l	Hour (1 to 12)
%M	Month name in full (January to December)
%m	Month name as a numeric value (00 to 12)
%p	AM or PM
%r	Time in 12 hour AM or PM format (hh:mm:ss AM/PM)
%S	Seconds (00 to 59)
%s	Seconds (00 to 59)
%T	Time in 24 hour format (hh:mm:ss)
%U	Week where Sunday is the first day of the week (00 to 53)
%u	Week where Monday is the first day of the week (00 to 53)
%V	Week where Sunday is the first day of the week (01 to 53). Used with %X
%v	Week where Monday is the first day of the week (01 to 53). Used with %x
%W	Weekday name in full (Sunday to Saturday)
%w	Day of the week where Sunday=0 and Saturday=6
%X	Year for the week where Sunday is the first day of the week. Used with %V
%x	Year for the week where Monday is the first day of the week. Used with %v
%Y	Year as a numeric, 4-digit value
%y	Year as a numeric, 2-digit value

Aggregate:

>used to summarize data

AVG	Returns the average value of an expression
COUNT	Returns the number of records returned by a select query
MAX	Returns the maximum value in a set of values

MIN	Returns the minimum value in a set of values
SUM	Calculates the sum of a set of values

Sub-Query:

>it is a query that is defined as part of main query.
>it is also known as nested query
>only select command can be used as sub-query
>outer query may be select,update,delete,etc.

case-1(with where):
---
exp:
select * from table where col=(select agg_fun() from table);

Note:
>table of sub-query can not be same as main query if main query is delete or update

exp:
update table set col=expression where col=(select agg_fun() from table) #error
delete from table where col=(select agg_fun() from table) #error

>this combination is allowed with different tables
exp:
update table1 set col=expression where col=(select agg_fun() from table2) #correct
delete from table1 where col=(select agg_fun() from table2) #correct

case-2(with from)
---
select * from (select col1,col2,..from table) as derived;

>in this case,result of sub-query acts as table for main query
>we must define alias of sub-query
>conceptually,result of sub-query is called derived table

case-3(with select)
---
select col1,col2,(select col from table2 where table1.matchcol=table2.matchcol) as alias from table1;

>It is like left join or vlookup in excel

>Only single col can be selected in subquery
>If you fetch anyaggregate value instead of col then same value is shown in each row.

group by

>GROUP BY is used to make groups of similar records based on a category
 and then do calculation on each group.

>We use it when we want summary information, not individual rows.

```
SELECT column_name, AGG_FUNCTION(column)
FROM table_name
GROUP BY column_name;
```

group by with where

```
SELECT dept, SUM(salary)
FROM employee
WHERE salary > 45000
GROUP BY dept;
```

group by with having

```
SELECT dept, SUM(salary)
FROM employee
GROUP BY dept
HAVING SUM(salary) > 100000;
```

where v/s having:
---

where	having
works with cols of table	works with selected cols & expr only (widely used with group by)

WHERE → before GROUP BY HAVING → after GROUP BY

Window Functions:

>A window function performs operations on a set of query rows(called windows).
>We use over() clause along with window functions
>if over() clause is empty,it means whole table is considered as 1 window

row_number():

>returns unique int for each row in given window.

>it always starts from 1 and incremented by 1

syntax-1:

select row_number() over() from table

it will assign row number from 1 to N according to PK,if PK is not defined then either in insertion or unorder manner

syntax-2:

select row_number() over(order by col asc/desc) from table

it will assign row number from 1 to N according to given order

syntax-3:

select row_number() over(partition by cat_col) from table

it will assign row number from 1 to N for each window

syntax-4:

select row_number() over(partition by cat_col order by col asc/desc) from table

it will assign row number from 1 to N for each window according to given order

rank():

>Returns the rank of the current row within its partition, with gaps.

>Peers are considered ties and receive the same rank.

>This function should be used with ORDER BY to sort partition rows into the desired order. Without ORDER BY, all rows are peers.

>select *,rank() over(order by empsal) as num from emp2;

>select *,rank() over(order by empsal desc) as num from emp2;

>select *,rank() over(partition by empdept order by empsal) as num from emp2;

dense_rank():

>Returns the rank of the current row within its partition, without gaps.

>Peers are considered ties and receive the same rank.

>select *,dense_rank() over(order by empsal) as num from emp2;

>select *,dense_rank() over(order by empsal desc) as num from emp2;

>select *,dense_rank() over(partition by empdept order by empsal) as num from emp2;

first_value():

>Returns the value of expr from the first row of the window frame.

>select *,first_value(empsal) over(partition by empdept order by empsal) as num from emp2;

last_value():
>Returns the value of expr from the last row of the window frame.
select *,last_value(empsal) over(partition by empdept order by empsal) as num from emp2;

lead():
>returns next value of col in the partition or null
>select *,lead(empsal) over() as n from emp2

lag()
>returns previous value of col in partitions or null

PL/SQL:
=======
>It is extension of SQL that focus on logic reusability along with SQL queries
>There are following 3 main concepts in PL/SQL
1. Function
2. Procedure
3. Trigger

Function

>It takes 0-N input as parameters and returns exactly 1 value

syntax:

```
create function fun_name(param1 datatype,param2 datatype,.) returns datatype characteristics
begin
local variable declartion;
logic
return expr;
end
```

>select command is used to run function
syntax:
```
select fun_name(value1,value2,...);
```

characteristics:
>control the access for tables
1. no sql :function can not run any sql query
2. reads sql data :function can run select query
3. modifies sql data :function can run DML queries

etc.

Procedure

>It takes 0-N input as parameters and also return 0-N result as out parameters

syntax:

```
create procedure procedure_name(in p1 datatype,in p2 datatype,out p3 datatype,out p4
..)
begin
logic
end
```

parameter type:

IN parameter

It is the default mode. It takes a parameter as input, such as an attribute. When we define it, the calling program has to pass an argument to the stored procedure. This parameter's value is always protected.

OUT parameters

It is used to pass a parameter as output. Its value can be changed inside the stored procedure, and the changed (new) value is passed back to the calling program. It is noted that a procedure cannot access the OUT parameter's initial value when it starts.

INOUT parameters

It is a combination of IN and OUT parameters. It means the calling program can pass the argument, and the procedure can modify the INOUT parameter, and then passes the new value back to the calling program.

How to call a stored procedure?

1>with IN parameter

```
CALL procedure_name ( parameter(s))
```

2>with out parameter

```
CALL procedure_name(@var);
SELECT @var;
```

3>with inout parameter

```
set @var=value
CALL procedure_name(@var);
SELECT @var;
```

>call command is used to run procedure

syntax:

```
call pro_name(v1,v2,@v3,@v4..);
select @v3,@v4;
```

@var is a session variable used to store result of out parameter

Note:

Both function & procedure are stored and can be used anytime.

Diff b/w function & procedure:

---

function	procedure
return exactly 1 value	can return 0-N values as out params
we have to specify characterstic	N/A
can not produce record set	can produce record set(tabular result)
best suitable for select	best suitable for DML
does not support txn	supports txn

Triggers:

>A trigger is a special type of stored procedure automatically executed by sql engine for DML operations.

>Generally,triggers are used to run background logic on DML operations like log,validation,etc.

Types of Triggers in MySQL

We can define the maximum six types of actions or events in the form of triggers:

Before Insert: It is activated before the insertion of data into the table.
After Insert: It is activated after the insertion of data into the table.

Before Update: It is activated before the update of data in the table.
After Update: It is activated after the update of the data in the table.

Before Delete: It is activated before the data is removed from the table.
After Delete: It is activated after the deletion of data from the table.

Syntax:

```
CREATE TRIGGER trigger_name
   (AFTER | BEFORE) (INSERT | UPDATE | DELETE)
       ON table_name FOR EACH ROW
       BEGIN
         trigger code
      END
```

The NEW and OLD modifiers(keywords) are used to distinguish the column values BEFORE and AFTER the execution of the DML statement.

We can use the column name with NEW and OLD modifiers as OLD.col_name and NEW.col_name.
The OLD.column_name indicates the column of an existing row before the updation or deletion occurs.
NEW.col_name indicates the column of a new row that will be inserted or an existing row after it is updated.

Trigger Event	OLD	NEW
INSERT	No	Yes
UPDATE	Yes	Yes
DELETE	Yes	No

How to show all functions:

```
show function status where db='db_name';
```

How to show all procedures:

```
show procedure status where db='db_name';
```

How to show all triggers:

```
    SELECT TRIGGER_NAME
   -> FROM information_schema.triggers
   -> WHERE trigger_schema = '1030am';
```

How to show logic of procedure/function/trigger

```
show create procedure/function/trigger name;
```

How to Backup a database

```
mysqldump --set-gtid-purged=OFF -u username -p database_name > filename.sql
```

How to Backup a table

```
mysqldump --set-gtid-purged=OFF -u root -p database_name table_name > filename.sql
```

How to Backup all database

```
mysqldump --set-gtid-purged=OFF -u root -p --all-databases > filename.sql
```

How to restore database

```
mysql -u root -p database_name < filename.sql
```

How to create user:

```
CREATE USER 'username'@'localhost' IDENTIFIED BY 'password';
```

How to change password:

```
ALTER USER 'username'@'localhost' IDENTIFIED BY 'new_password';
```

Note:not recommended with root user

Grant all privileges on a specific database

GRANT ALL PRIVILEGES ON database_name.* TO 'new_username'@'localhost';
*-->means all tables

Revoke privileges

REVOKE SELECT, INSERT ON database_name.* FROM 'username'@'localhost';
REVOKE ALL PRIVILEGES ON database_name.* FROM 'username'@'localhost';

How to login with other user:

1.set path of mysql bin directory
2.open cmd/terminal and write cmd
mysql -u user -p

MySQL Editors:
1.Command line editor (CUI)
2.Workbench editor (GUI)

you can download workbench editor from
https://dev.mysql.com/downloads/workbench/